GRADUATION PROJECT DOCUMENTATION • PARTS 1–3

EASYMENU AI

AI-Powered Menu Extraction, Restaurant Analytics Chatbot, and Food Image Generation

Project Planning & Management
Literature Review
Requirements Gathering

Digital Egypt Pioneers Initiative (DEPI)
Machine Learning Engineer Track

Team Leader
Mohaned Akl

Team Members
Ahmed Abdelstar Diab Shehata
Al-Baraa Hesham Fouad Mahmoud
Lojain Hussein Saber Mohamed
Mariam Waleed Mohamed Gharib Attia

Repository: https://github.com/mohanadakl/EasyMenu-AI
Version 1.0 — 19 October 2026
Team draft for mentor review. Describes proposed work; no results are claimed.

Contents

1. Project Planning & Management . . . 3

1.1 Project Proposal . . . 3

1.2 Project Objectives . . . 4

1.3 Project Scope . . . 5

1.4 Project Methodology . . . 5

1.5 Project Plan and Timeline . . . 6

1.6 Milestones and Deliverables . . . 7

1.7 Resource Allocation . . . 7

1.8 Task Assignment & Roles . . . 8

1.9 Risk Assessment & Mitigation Plan . . . 8

1.10 Key Performance Indicators (KPIs) . . . 9

2. Literature Review . . . 10

2.1 Review Approach . . . 10

2.2 Menu Extraction from Images and PDFs . . . 10

2.3 Natural-Language Analytics and Text-to-SQL . . . 11

2.4 Food Image Generation . . . 12

2.5 Comparative Analysis . . . 13

2.6 Research Gap and Project Contribution . . . 14

2.7 Feedback & Evaluation . . . 14

2.8 Suggested Improvements and Proposed Grading Criteria . . . 15

3. Requirements Gathering . . . 16

3.1 Stakeholder Analysis . . . 16

3.2 User Stories . . . 17

3.3 Use Cases . . . 17

3.4 Functional Requirements . . . 19

3.5 Non-functional Requirements . . . 20

3.6 System Constraints, Assumptions, and Dependencies . . . 20

3.7 Requirements Prioritization . . . 21

3.8 Requirements Traceability Matrix . . . 21

Appendix A. Items Requiring Team Confirmation . . . 22

References . . . 23

1. Project Planning & Management

1.1 Project Proposal

EasyMenu is an existing restaurant management system. **EasyMenu AI** is the graduation-project extension that investigates, develops, and evaluates Deep Learning components intended to replace or reduce selected third-party AI/API dependencies in three restaurant workflows: extracting structured data from menu images and PDF files, answering business questions in natural language through safe database queries, and generating representative food images from item names and descriptions. The project is carried out within the DEPI Machine Learning Engineer track.

> **Document status**
> This document describes the project context and **proposed** work. No model training, integration, or evaluation results are claimed. The project repository README likewise describes the three modules as proposed and not yet implemented or validated. Facts that the team still has to confirm are collected in Appendix A.

Item	Details
Project name	EasyMenu AI
Context	Graduation project of the Digital Egypt Pioneers Initiative (DEPI), Machine Learning Engineer track
Repository	https://github.com/mohanadakl/EasyMenu-AI
Project type	Applied Deep Learning and system integration, extending the existing EasyMenu restaurant management system
Team	Five members led by Mohaned Akl (see Section 1.8)
Documentation scope	Part 1 Project Planning & Management, Part 2 Literature Review, Part 3 Requirements Gathering
Document version	1.0, dated 19 October 2026 (team draft for mentor review)

Background and problem statement

Restaurant operators routinely need to enter menu data, understand what their operational data says, and prepare visual content for menu items. External AI APIs can speed these tasks up, but they may bring usage costs, dependence on third-party availability, data-transfer considerations, and limited control over model behaviour. EasyMenu AI explores project-managed Deep Learning components so that the team can understand, evaluate, and control the behaviour of each capability. The table below states the problem addressed by each workflow.

Workflow	Problem	Proposed AI capability
A. Menu data entry	Menus arrive as photographs or PDF files with varied layouts. Typing sections, item names, prices, and descriptions by hand is slow and error-prone.	AI-powered menu extraction that converts an image or PDF into structured data for human review.
B. Business insight	Answers about sales or operations normally require database queries or dashboards that non-technical administrators cannot write themselves.	Restaurant analytics chatbot that turns a question into a validated, read-only SQL query and explains the result.
C. Menu visuals	Menu items need representative images, which require photography, stock imagery, or paid generation services.	AI food image generation from an item name and description, with review before use.

Proposed solution

The proposed solution adds three AI services around the existing EasyMenu system. Each service exposes a documented interface to EasyMenu and places a human review or validation gate before data is saved or a query is executed (Figure 1). The exact interfaces, model choices, and deployment arrangement are not yet fixed and will follow the baseline experiments described in the plan.

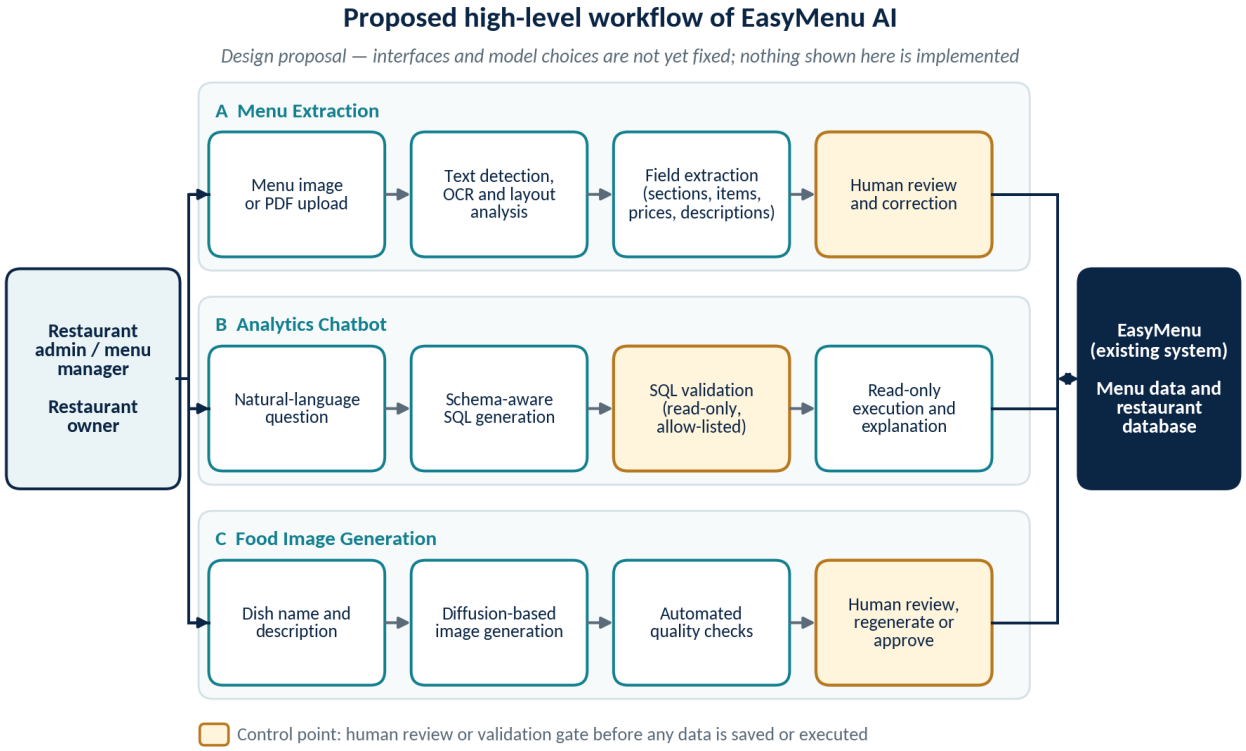


Figure 1. Proposed high-level workflow. Lane B reads authorized data from EasyMenu through a read-only connection; lanes A and C return only reviewed outputs.

Target users and beneficiaries

The primary users are restaurant administrators and menu managers, who upload menus and request images, and restaurant owners or managers, who ask analytics questions. Indirect beneficiaries are customers who read the menu, and system administrators who keep database access safe. Section 3.1 analyses all stakeholders in detail.

1.2 Project Objectives

- **O1 Menu extraction.** Design and evaluate a pipeline that extracts menu sections, item names, prices, and descriptions from menu images and PDF files into structured data suitable for review.
- **O2 Analytics chatbot.** Develop or adapt a language-model-based assistant that translates supported natural-language questions into validated, read-only SQL and explains the results.
- **O3 Food image generation.** Evaluate and, where feasible, adapt a diffusion model to generate representative food images from item names and descriptions.
- **O4 Integration.** Integrate the components with EasyMenu through documented interfaces, subject to feasibility and available compute resources.
- **O5 Safety and quality.** Evaluate correctness, safety, latency, and robustness with documented test cases, and report limitations honestly.
- **O6 Deliverables.** Deliver the DEPI documentation set, source code on GitHub, a test report, a demonstration, a final presentation, and the project infographic by the official dates in Section 1.5.

1.3 Project Scope

In-scope features

Component	In-scope features
A. Menu extraction	Upload of menu images and PDF files, with clear errors for unsupported or unreadable files. Extraction of menu sections, item names, prices, and descriptions when present; other supported fields (for example currency or size variants) are proposed and optional. Structured output with uncertainty indicators where the selected pipeline supports them. Review and correction by a person before anything is saved to the menu.
B. Analytics chatbot	Natural-language questions about authorized restaurant business data within a defined supported scope. Schema-aware SQL generation, validation, and execution restricted to read-only operations. Natural-language explanation of the returned result, and a clear message when a question cannot be answered reliably.
C. Food image generation	Image generation from an item name and description, with a clear failure message when generation fails. Review, regeneration, or rejection of generated images; labelling as AI-generated illustrations. Optional adaptation (for example LoRA) of the selected model if data, licence, and compute allow.
Cross-cutting	Documented interfaces for integration with EasyMenu, error handling, and event logging. Evaluation datasets and test cases, a test plan and report, setup instructions, and a demonstration.

Out-of-scope features

- Rebuilding the EasyMenu application or replacing unrelated existing features.
- Training a foundation model from scratch.
- Guaranteeing production-scale performance or commercial-grade accuracy before evaluation.
- Executing any data-modifying SQL from the analytics chatbot, or saving extracted menu data without review.
- Presenting generated images as photographs of the dish that is actually served.
- Handwritten or heavily stylised menu artwork in the first iteration (proposed; to be confirmed).

Optional and future work (proposed)

- Additional menu languages beyond those confirmed by the team (Arabic and English are candidates; see Appendix A).
- Model compression or quantisation to reduce inference cost.
- Charts and multi-turn conversation in the analytics chatbot.
- Style consistency across all generated images of a menu, using restaurant-provided reference photos.

1.4 Project Methodology

The team proposes an iterative, baseline-first methodology with interfaces defined early and a human in the loop for every output that changes restaurant data. Work is organised in weekly sprints with a short gate review at the end of each week. Collaboration follows the version-control expectations of the DEPI guidelines: a GitHub repository, a clear branching strategy, meaningful commit messages, and pull-request descriptions.

Phase	Activities	Output
1. Define	Confirm scope, requirements, evaluation criteria, and interface contracts for the three components.	Approved documentation and interface drafts
2. Baseline	Run pretrained or off-the-shelf candidates on representative data; record metrics and failure cases.	Baseline results and model-selection rationale
3. Adapt	Fine-tune or adapt selected models where feasible (for example parameter-efficient methods), within licence and compute limits.	Adapted models or documented configurations
4. Integrate	Expose services through documented interfaces, add validation and review steps, and connect to EasyMenu.	Integrated prototype
5. Verify	Run the test plan, analyse failures, and document limitations.	Test report and final documentation

1.5 Project Plan and Timeline

The schedule below is a **proposed plan**. It is anchored to the official DEPI submission dates and requires confirmation by the team and mentor. Work planned for future dates is not complete, and no milestone after Part 1–3 documentation is claimed as done.

Official DEPI submission dates

Deliverable (per DEPI guidelines)	DEPI date
Project Planning & Management	16 Oct 2026
Literature Review	16 Oct 2026
Requirements Gathering	16 Oct 2026
System Analysis & Design	6 Nov 2026
Implementation (Source Code & Execution)	30 Nov 2026
Final Presentation, Testing & Reports	4 Dec 2026

This document is dated 19 October 2026, the team's documentation deadline. The guidelines list 16 October 2026 for the first three items; the team should confirm which date applies (Appendix A).

Proposed sprint calendar

Week	Dates	Proposed focus	Gate or official date
W1	19 – 25 Oct	Submit Parts 1–3; fix interface contracts; set up repository structure; review data and model licences.	Parts 1–3 (19 Oct)
W2	26 Oct – 1 Nov	Prepare datasets and test sets; run baseline inference for each component; draft system design.	Baselines started
W3	2 – 8 Nov	Submit System Analysis & Design; evaluate baselines; choose models.	Design (6 Nov)
W4	9 – 15 Nov	Adapt models where feasible; build the three services and their validators.	Services running locally
W5	16 – 22 Nov	Integrate with EasyMenu; build review screens; safety and error-handling tests.	Integrated prototype
W6	23 – 29 Nov	Integration testing and evaluation runs; proposed feature freeze on 29 Nov.	Implementation (30 Nov)
W7	30 Nov – 6 Dec	Final testing, reports, presentation, infographic, and demonstration rehearsal.	Final (4 Dec)

Proposed Gantt chart

Work package	Window	W1	W2	W3	W4	W5	W6	W7
WP0 Documentation Parts 1–3	W1	■						
WP1 Data and licence review	W1–W2	■	■					
WP2 Baselines and model selection	W2–W3		■	■				
WP3 System analysis and design	W1–W3	■	■	■				
WP4 Model adaptation and evaluation	W3–W5			■	■	■		
WP5 Services and EasyMenu integration	W4–W6				■	■	■	
WP6 Testing and quality assurance	W5–W7					■	■	■
WP7 Presentation, infographic, reports	W6–W7						■	■

Shaded cells show the weeks in which each work package is proposed to be active. W1 starts on 19 October 2026; the official final date is 4 December 2026.

1.6 Milestones and Deliverables

Milestone	Target date	Deliverables	Acceptance criteria
M1 Planning documentation	19 Oct 2026	Project Planning & Management, Literature Review, and Requirements Gathering (this document); repository link.	Reviewed by the team; mentor confirmation pending.
M2 Data and baselines	1 Nov 2026 (proposed)	Dataset and licence notes; test sets; baseline results for the three components.	Baselines reproducible; metrics defined in Section 1.10 recorded.
M3 System Analysis & Design	6 Nov 2026 (official)	Use cases, architecture, data model, data-flow and behaviour diagrams, UI mock-ups, technology stack.	Design consistent with Part 3 requirements.
M4 Adapted models	22 Nov 2026 (proposed)	Selected or adapted models, configurations, and evaluation report.	Results reported against the proposed KPI targets, including failures.
M5 Implementation	30 Nov 2026 (official)	Source code, README, integrated prototype, interface documentation.	End-to-end flows of the three components run on the demo setup.
M6 Final presentation and reports	4 Dec 2026 (official)	Test plan and report, user manual, technical documentation, presentation, infographic, demonstration.	Documents complete and uploaded to GitHub.

1.7 Resource Allocation

The resources below are **candidates** drawn from the project concept. Final choices depend on experiments, compute, data quality, and licensing.

Resource	Planned use	Proposed owner	Status
GitHub repository	Version control, documentation, issue tracking, branching and pull requests	Team leader	Repository exists
Python and a deep learning framework (PyTorch or TensorFlow)	Training, adaptation, and inference code	All members	Candidate
PaddleOCR, LayoutLMv3, Donut	Candidate OCR, layout, and structured-extraction approaches for Component A	Component A owner	To be evaluated
Open language models (candidates include Code Llama, Qwen, Llama)	Candidate Text-to-SQL models for Component B	Component B owner	To be evaluated
Stable Diffusion XL, optional LoRA	Candidate image generation and adaptation for Component C	Component C owner	To be evaluated

Resource	Planned use	Proposed owner	Status
GPU compute	Adaptation and inference experiments	Team leader	Pending confirmation
EasyMenu codebase and database schema	Integration and analytics test data	Team leader	Pending access confirmation
Datasets (menus, question–SQL pairs, food images)	Training, adaptation, and evaluation	Component owners	Sources and licences pending

1.8 Task Assignment & Roles

The role allocation below is a **proposed starting point** based on the team's initial plan. It is not confirmed and no past contribution is asserted. Final responsibilities should be agreed with the mentor and updated if the actual work distribution changes. Every member is expected to contribute to documentation and the final presentation.

Team member	Proposed role	Proposed primary responsibilities	Proposed cross-review
Mohaned Akl	Team Leader and Project Integration	Scope and milestones; integration with EasyMenu; interface contracts; repository management; final submission.	Component B
Al-Baraa Hesham Fouad Mahmoud	Menu Extraction / OCR	Menu samples; OCR and document-extraction pipeline; extraction evaluation.	Component C
Ahmed Abdelstar Diab Shehata	Analytics Chatbot / Text-to-SQL	Schema-grounded SQL generation; query validation and safety tests; analytics test cases.	Component A
Mariam Waleed Mohamed Gharib Attia	Food Image Generation / Model Evaluation	Model and licence review; image–caption data; adaptation and image evaluation.	Component B
Lojain Hussein Saber Mohamed	Testing, Requirements and Documentation	Requirements; test plans and reports; reproducibility notes; documentation; demonstration support.	Components A and C

1.9 Risk Assessment & Mitigation Plan

Probability and impact are the team's initial judgement (Low, Medium, High) and should be revisited at each weekly gate. Risks are planning considerations, not reports of incidents.

Risk	Prob.	Impact	Mitigation	Contingency
Compressed schedule against fixed official dates	High	High	Baseline-first plan; weekly gate reviews; interface-first development; proposed feature freeze on 29 Nov.	Keep pretrained baselines with prompt and schema engineering; defer fine-tuning.
Insufficient or unrepresentative data	Medium	High	Use permitted representative samples; document limitations; split data carefully by menu, schema, and dish.	Report results on a smaller test set with stated limits.
Limited GPU compute	High	Medium	Start with inference baselines and parameter-efficient adaptation; reduce resolution and batch size.	Use approved compute or smaller model variants.
OCR or layout errors, especially on complex or multilingual menus	High	High	Confidence and validation checks; test varied layouts; mandatory human review before saving.	OCR-only baseline with rule-based assembly, then manual correction.
Incorrect or unsafe generated SQL	Medium	High	Read-only database account; schema allow-list; SQL parser and validator; row and time limits; audit log.	Restrict to a fixed set of query templates.
Unauthorized data access or prompt injection through the chatbot	Medium	High	Authorization enforced by the host system; restricted views; treat question and data text as untrusted input.	Disable free-form queries and expose pre-approved questions only.
Generated image misleads about the actual dish	Medium	Medium	Label as AI-generated; human approval; prompt-adherence review.	Allow upload of a real photograph instead.

Risk	Prob.	Impact	Mitigation	Contingency
Unclear model or data licensing	Medium	High	Maintain a licence register; review each checkpoint and dataset before use (see Section 2).	Switch to an alternative permitted model or dataset.
Integration incompatibility with existing EasyMenu	Medium	High	Define interface contracts early; stub endpoints; test components independently.	Demonstrate through a standalone interface.
Privacy of restaurant data	Medium	High	Use synthetic or anonymised data for tests; keep secrets and private data out of Git.	Use sample data only.
Team availability and unconfirmed roles	Medium	Medium	Weekly planning; cross-review between components; confirm roles with the mentor.	Reassign tasks and protect the core flows.
Scope creep	Medium	Medium	Prioritise the three stated workflows; defer optional features until core evaluation is complete.	Cut optional features.

1.10 Key Performance Indicators (KPIs)

All numeric values are **proposed targets to be validated** after baseline measurements and agreed with the mentor. They are not results. Where a target is not yet justified, the KPI is reported rather than targeted.

KPI	Proposed target (to be validated)	How it is measured
Menu field extraction quality (item name, price)	Field-level F1 of at least 0.85 on held-out menus	Precision, recall, and F1 against manually labelled menus, with normalised matching.
Price exact match	At least 0.90 after normalisation	Exact match of normalised price values.
Section assignment accuracy	At least 0.80	Share of items assigned to the correct section.
Invalid-file handling	100% of invalid-file test cases give a clear error	Test cases with unsupported, corrupt, and unreadable files.
Correction effort	Reported, no target	Median number of fields a reviewer corrects per menu.
Text-to-SQL execution accuracy	At least 0.75 on the supported-question test set	Share of questions whose validated query returns the expected result.
SQL safety	0 write or unauthorized statements executed in the safety suite	Destructive and unauthorized test attempts, with blocked and executed counts.
Explanation faithfulness	At least 90% of reviewed answers consistent with the result set	Manual review of sampled answers.
Image prompt adherence	Mean rubric score of at least 3.5 out of 5	Human rubric: dish identity, attributes, quality, and artefacts.
Image acceptability	At least 70% judged acceptable for menu use	Reviewer vote on a representative prompt set.
Response time	Median and 95th percentile reported per workflow; targets set after baseline	End-to-end timing on documented hardware.
Reliability	At least 95% of valid test inputs complete without an unhandled error	Test-run log.
Integration completeness	All must-have acceptance tests passing; report passed/total	Acceptance-test results.
Reproducibility	100% of reported experiments record model version, configuration, and data split	Experiment log audit.
On-time delivery	All official DEPI dates met	Weekly gate-review log.

The DEPI guidelines cite response time, system uptime, and user adoption as example KPIs. Response time is covered above. Uptime applies only to the demonstration environment, and user adoption is not applicable because a production rollout is outside the project scope.

2. Literature Review

2.1 Review Approach

This review examines published research and official open-source resources relevant to the three planned AI components. It informs model selection. It does **not** imply that any cited model has been integrated, trained, or compared by the EasyMenu AI team.

- Sources are original papers, official repositories, and official model documentation, organised around the three problems the project addresses.
- Every reference in the References section was looked up in public sources on 19 October 2026; the verification status of each is stated there.
- Statements are separated into what published work reports, what the team proposes as a design decision, and what still needs testing (Section 2.6).
- Comparison tables are qualitative judgements based on published descriptions. They contain no experimental results from the team.

2.2 Menu Extraction from Images and PDFs

The problem

Extracting a menu is a document-understanding task with four stages: detecting and recognising text (OCR), analysing layout and reading order, assigning text to fields such as section, item name, description, and price, and normalising the result into a consistent schema. Menus make each stage harder than typical forms: multi-column pages, section headings, leader dots between an item and its price, several prices per item for different sizes, decorative fonts, mixed languages, and photographs with perspective distortion or glare. OCR alone returns text and positions; it does not decide which price belongs to which item. A PDF may contain embedded text or may be a scanned image, and the project has not yet confirmed which case applies (Appendix A).

Candidate approaches

Approach	What it addresses	Strengths	Limitations for this project
PaddleOCR [1][2]	An open-source toolkit for OCR and document parsing. Version 3.0 presents PP-OCRv5 for multilingual text recognition, PP-StructureV3 for hierarchical document parsing, and PP-ChatOCRv4 for key information extraction [2].	Modular, inspectable stages; Apache-licensed per its technical report [2]; the repository's release notes list newer components with broad language coverage, including Arabic in the vision-language parsing model [1].	Language and layout claims come from the project's own documentation and were not tested on menus. Assigning text to menu fields still needs extra logic or a trained extraction model.
LayoutLMv3 [3][4]	A multimodal Transformer pre-trained with unified text and image masking, reported to perform strongly on form understanding, receipt understanding, document question answering, and layout analysis [3].	Combines text, 2D layout, and visual appearance, which helps when position decides meaning, such as price versus item name [3].	Works on top of OCR output (words and boxes), so OCR errors propagate. Needs labelled menu data for fine-tuning. The model card lists English and a non-commercial CC BY-NC-SA 4.0 licence; the licence field has changed in the card's history, so it must be re-checked before use [4].
Donut [5]	An OCR-free Transformer that maps a document image directly to structured output, motivated by OCR cost, inflexibility across languages and document types, and OCR error propagation [5].	Removes the separate OCR stage; the authors provide a synthetic data generator intended to help pre-training for different languages and domains [5].	Needs paired image-to-structure training examples. Generated output must be validated because missing or invented fields are possible. Dense multi-column pages may need tiling or high input resolution (to be tested).

Images, PDFs, and multilingual content

For photographed menus, image quality (skew, glare, low resolution) is the main risk, so pre-processing and a confidence check are part of the proposed design. For PDFs, a born-digital file could be read through its embedded text and layout, while a scanned PDF must be rasterised and processed like an image; multi-page documents also need page-level processing and merging. For multilingual content, each candidate must be tested on the languages the team confirms, because the sources describe language coverage in general terms and do not evaluate menus. Fine-tuning data in the target languages will probably be needed for any learned extractor.

Proposed structured representation

Whatever model is selected, its output would be converted into one consistent record so that review, validation, and integration do not depend on the model. The fields below are a **proposed schema**; the final schema must match the EasyMenu data model (Appendix A).

Field	Description
section_name	Menu section or category that groups the item.
item_name	Product name as printed.
description	Optional description or ingredients text.
price	Normalised numeric price; several entries for size variants where present.
currency	Currency symbol or code, when stated.
confidence / flags	Model confidence and flags for missing or uncertain fields, where the pipeline supports them.
source_page and location	Page number and region of the source file, so a reviewer can compare against the original.
review_status	Pending, corrected, or approved; only approved records are saved.

Evaluation

- OCR stage: character error rate and word error rate against transcribed text.
- Field stage: precision, recall, and F1 for item name, price, section, and description, using exact and normalised matching.
- Structure: share of items assigned to the correct section; correct handling of multi-price items.
- Usability: number of fields a reviewer must correct per menu, and results broken down by layout type, language, and image quality.

2.3 Natural-Language Analytics and Text-to-SQL

The problem

Text-to-SQL translates a question into a database query. The difficulty lies in schema awareness: linking words in the question to the correct tables and columns (schema linking), choosing joins, handling aggregation and date filters, and coping with ambiguity. For restaurant analytics, a question about sales over a period must map to the right tables and filters of the actual EasyMenu schema, which the team has not yet confirmed (Appendix A). Because generated SQL runs against business data, correctness and safety are equally important.

Benchmarks and approaches

- **Spider [6]** is a cross-domain Text-to-SQL benchmark of 10,181 questions and 5,693 unique complex SQL queries over 200 multi-table databases in 138 domains. Its train and test sets use different databases, so it measures generalisation to unseen schemas, which is also the situation of a model facing a new restaurant schema.
- **RAT-SQL [7]** addresses schema encoding and schema linking with relation-aware self-attention, to help parsers generalise to unseen database schemas. It is a research parser and its relevance to EasyMenu is the design idea of modelling question-to-schema relations explicitly.
- **Code Llama [8]** is a family of code-specialised language models built on Llama 2, offered as base, Python, and instruction-following variants in 7B to 70B sizes. Its headline results are on code-generation benchmarks (HumanEval, MBPP, MultiPL-E), not Text-to-SQL, so suitability for SQL over the EasyMenu schema must be tested.

- Other open model families named in the project README, such as Qwen and Llama, were not reviewed here and need separate verification of capability and licence.

Safety and reliability

The principal risks are incorrect queries, hallucinated tables or columns, unauthorized data access, destructive statements, expensive queries, and answers that do not match the returned data. A layered design is proposed. Database permissions are the authoritative control; the SQL validator is defence in depth, not a substitute.

Risk	Proposed control
Destructive or data-modifying SQL	Database role with read-only privileges; validator accepts a single read-only statement and rejects everything else.
Hallucinated or unauthorized tables and columns	Schema allow-list or restricted views; validation of every identifier against the allowed schema.
Access to another restaurant's or user's data	Authorization enforced by the host system and scoped queries; to be aligned with the EasyMenu data model.
Expensive or runaway queries	Statement timeout, row limit, and result-size limit.
Prompt injection through question or data text	Treat user text and database content as untrusted; keep instructions separate from data; never execute SQL that fails validation.
Answer inconsistent with results	Generate the explanation only from the returned rows; state clearly when a question cannot be answered reliably.
No traceability	Audit log of questions, generated SQL, validation outcome, and errors, without storing secrets.

Evaluation

- Execution accuracy: whether the validated query returns the expected result on a team-built set of questions, gold SQL, and expected results over the supported tables. Exact string match is less informative because different SQL can be equivalent.
- Valid-SQL rate and hallucinated-identifier rate.
- Safety suite: destructive, unauthorized, and injection-style attempts, reporting blocked versus executed counts.
- Correct refusal of out-of-scope or ambiguous questions; end-to-end latency; and manual review of explanation faithfulness.

2.4 Food Image Generation

Relevant techniques

- **Denoising diffusion probabilistic models [9]** are generative models that learn to reverse a gradual noising process; the work reports high-quality image synthesis and connects the training objective to denoising score matching.
- **Latent diffusion models [10]** run diffusion in the latent space of a pretrained autoencoder and add cross-attention so that text can condition generation, reducing computational requirements compared with pixel-space diffusion.
- **Stable Diffusion XL [11][12]** is a latent diffusion model for text-to-image synthesis with a three times larger UNet backbone, a second text encoder, additional conditioning schemes, multi-aspect-ratio training, and an optional refinement model [11]. The model card states the CreativeML Open RAIL++-M licence, which permits use subject to use-based restrictions, and lists limitations: imperfect photorealism and no legible text rendering [12].
- **LoRA [13]** freezes pretrained weights and injects trainable low-rank matrices, greatly reducing trainable parameters and GPU memory. The paper evaluates language models. Applying the idea to diffusion models is a later adaptation that this review does not cover, so the team must confirm tooling support and results in practice.

Relationship to food images, data needs, and risks

A pretrained text-to-image model can be prompted with a dish name and description to produce a plausible image, but general pretraining does not guarantee that a regional dish looks correct. Adaptation, if attempted, needs image–caption pairs of dishes with consistent framing and style, accurate captions, clear licences or consent, and a split by dish so that test images do not leak into training. The dataset size, source, and cultural coverage are not yet defined (Appendix A). Principal risks are loss of fidelity to the real dish (ingredients, portions), low visual diversity or inconsistent style across a menu, cultural or other bias inherited from pretraining data, artefacts, and inference cost and latency. The SDXL model card also notes that the model is not trained to produce factual representations [12], which is why the proposal labels outputs as AI-generated illustrations.

Evaluation

- Human rubric (1–5), scored by at least two reviewers: dish identity, ingredient plausibility, adherence to the prompt, visual quality, artefacts, and cultural plausibility, plus inter-reviewer agreement.
- Acceptance rate: share of images a reviewer would approve for menu use, and number of regenerations needed.
- Automatic metrics such as FID or CLIP-based similarity may be used as secondary diagnostics only. They do not establish that an image shows the requested dish, and the SDXL paper itself discusses the limits of FID assessment for text-to-image models [11].
- Latency and GPU memory per image on documented hardware.

2.5 Comparative Analysis

The tables below compare plausible approaches for each component. Ratings are **qualitative design judgements** based on the published descriptions above; the team has run no benchmark, and no approach is claimed to be superior.

A. Menu extraction

Criterion	PaddleOCR-based modular pipeline	LayoutLMv3 on OCR output	Donut (OCR-free)
Suitability for the task	Good for text and layout; field assignment needs extra logic.	Good when position decides meaning; needs OCR first.	Direct image-to-structure; suits end-to-end extraction.
Accuracy potential	Medium to high, depends on assembly logic.	High if enough labelled data exists.	High if enough paired data exists.
Data requirements	Low for OCR; labelled data only to tune assembly.	Medium to high (token-level labels).	Medium to high (image–JSON pairs).
Compute	Low to medium.	Medium.	Medium to high.
Integration complexity	Medium, several stages.	Medium to high, OCR plus model.	Medium, one model, validation needed.
Main limitations	Rule brittleness on unusual layouts.	OCR error propagation; licence and language to verify.	Possible invented or missing fields; resolution limits.
Fit for project scope	Strong as a baseline.	Possible if labelled menus are available.	Possible; riskier within the schedule.

B. Analytics chatbot

Criterion	Rule or template mapping	Code-oriented LLM with schema context	Specialised schema-encoding parser
Suitability for the task	Safe for a narrow set of questions.	Handles varied phrasing.	Designed for unseen schemas.
Accuracy potential	High within its templates, none outside.	Medium to high; needs validation.	Medium to high; depends on training data.
Data requirements	Low.	Low for prompting; medium for fine-tuning.	High (question–SQL pairs).
Compute	Very low.	Medium to high for larger models.	Medium.
Integration complexity	Low.	Medium.	High.
Main limitations	Cannot generalise to new questions.	Hallucinated columns; unsafe SQL if unvalidated.	Research code; training effort.
Fit for project scope	Strong fallback.	Strong primary candidate with validators.	Useful as a design idea; less likely to be adopted whole.

C. Food image generation

Criterion	Pretrained SDXL, prompting only	SDXL with LoRA adaptation	Full fine-tuning of a diffusion model
Suitability for the task	Plausible images from text.	Better consistency for target dishes and style.	Highest control, highest cost.
Accuracy potential	Medium; may misrepresent regional dishes.	Medium to high with good data.	Potentially highest, higher overfitting risk.
Data requirements	None.	Curated image–caption set.	Large curated set.
Compute	Inference only.	Moderate training, inference.	High.
Integration complexity	Low.	Medium.	High.
Main limitations	Fidelity, style drift, licence terms.	Data licensing; tooling to confirm.	Compute, time, and licence terms.
Fit for project scope	Strong baseline.	Feasible if compute and data allow.	Unlikely within the schedule.

2.6 Research Gap and Project Contribution

The sources reviewed treat OCR and document understanding, Text-to-SQL, and image generation as separate research problems. They do not describe a restaurant-management workflow that combines the three with human review, read-only data access, and local control over model behaviour. This review is not a systematic survey, so the team makes no claim that no such system exists. EasyMenu AI is positioned as an **applied integration project**: its contribution is task-specific adaptation, safety controls, integration, and evaluation, not a new foundation-model architecture.

Supported by published research	Proposed design decisions of this project	Future work that needs testing
OCR and layout models exist as open-source tools [1][2][3][5]. Schema linking is central to Text-to-SQL generalisation [6][7]. Latent diffusion and LoRA reduce the cost of generation and adaptation [10][13].	Human review before saving extracted data. Read-only, validated SQL with database-level least privilege. Image labelling and approval before use.	Which OCR or extraction model suits EasyMenu menus. Whether the selected language model is accurate enough on the EasyMenu schema. Whether LoRA adaptation improves dish fidelity within compute limits.

2.7 Feedback & Evaluation

Supervisor or mentor feedback is pending. No feedback has been provided in the source files, and none is invented here. The table will be completed after the mentor review.

Area	Evidence to review	Mentor feedback	Status
Problem and scope	Sections 1.1–1.3	Not yet received	Pending

Area	Evidence to review	Mentor feedback	Status
Technical approach	Sections 1.4 and 2	Not yet received	Pending
Plan and risks	Sections 1.5–1.9	Not yet received	Pending
Evaluation and KPIs	Section 1.10	Not yet received	Pending
Requirements	Section 3	Not yet received	Pending

2.8 Suggested Improvements and Proposed Grading Criteria

Suggested improvements (team-proposed)

These are the team's own proposals for strengthening the project, not mentor feedback.

- Complete baselines for every component before attempting adaptation, so that any fine-tuning gain can be measured.
- Define the interface contracts with EasyMenu in the first two weeks so that the three tracks progress in parallel.
- Build evaluation sets early: labelled menus, question–SQL pairs, and an image-review rubric.
- Keep the human review gate and the read-only restriction as non-negotiable requirements.
- Maintain a model and data licence register and record every experiment's model version, configuration, and data split.

Proposed final grading criteria

The breakdown below is an **internal proposed rubric** aligned with the guideline wording. The weights are a working assumption and not the official grading policy; they should be replaced if an official rubric is supplied.

Category	Proposed weight	Evidence
Documentation and planning	20%	Planning, literature review, requirements, risks, system design
Implementation	40%	Source code, services, integration with EasyMenu, repository quality
Testing and evaluation	20%	Test plan and cases, metrics, error analysis, limitations
Presentation and demonstration	20%	Technical explanation, demonstration, individual contribution, Q&A

3. Requirements Gathering

Requirements are derived from the project concept, the intended users, and the DEPI guidelines. Features marked *Could* are proposed and optional. Priorities follow the MoSCoW scheme (Must, Should, Could). No requirement describes work that has already been completed.

3.1 Stakeholder Analysis

Stakeholder	Role and interest	Key needs	Priority	Success signal
Restaurant owner or manager	Reviews business performance through analytics.	Correct, understandable answers; no exposure of data to others.	High	Obtains a trustworthy answer without writing SQL.
Restaurant admin or menu manager	Creates and maintains the menu.	Fast menu entry; control over what is saved; usable item images.	High	Menu is created from a file with few corrections.
Restaurant staff	Work with menu content day to day.	Accurate names and prices.	Medium	No wrong prices reach the live menu.
Customers (indirect)	Read the menu and its images.	Accurate, clear, honestly labelled content.	Medium	Images do not mislead about the dish.
System or database administrator	Maintains the database and deployment.	Least-privilege access; logs; stable integration.	High	No unauthorized or destructive queries reach the database.
Project team (5 members)	Builds and documents the system.	Clear scope, reproducible experiments, modular code.	High	All three components integrate and are evaluated.
Supervisor, mentor, and evaluators	Review academic and technical quality.	Honest evaluation; traceable decisions; documentation.	High	Meets the DEPI criteria.

3.2 User Stories

ID	User story	Priority	Acceptance criteria
US-01	As an admin, I want to upload a menu image or PDF, so that I do not type the menu by hand.	Must	Supported image and PDF files are accepted and processed.
US-02	As an admin, I want to see extracted sections, items, prices, and descriptions next to the source, so that I can check them.	Must	Extracted fields are shown with their source page and any uncertainty flags.
US-03	As a menu manager, I want to correct and approve extracted data before saving, so that OCR mistakes do not become live menu data.	Must	Nothing is saved until the reviewer approves; corrections are kept.
US-04	As an admin, I want a clear message when a file is unsupported or unreadable, so that I know what to fix.	Must	Invalid files produce an understandable error and no partial save.
US-05	As a restaurant owner, I want to ask a business question in plain language, so that I can understand my data without SQL.	Must	A supported question returns an answer based on authorized data.
US-06	As an owner, I want the answer explained in understandable language, so that I can act on it.	Must	The explanation matches the returned result.
US-07	As an owner, I want to be told when a question cannot be answered reliably, so that I am not given a wrong answer.	Must	Out-of-scope or unsafe requests are refused with a clear message.
US-08	As a system administrator, I want generated queries restricted and validated, so that the chatbot cannot modify or expose unauthorized data.	Must	Only read-only, validated queries execute; blocked attempts are logged.
US-09	As a menu manager, I want to request an image from a dish name and description, so that I can prepare a menu visual.	Must	An image or a clear failure message is returned.
US-10	As a menu manager, I want to regenerate or reject an image, so that only a suitable image is used.	Should	Only an approved image is attached to the item.
US-11	As a customer, I want generated images identified as illustrations, so that I am not misled.	Should	Generated images carry an AI-generated label.
US-12	As a team member, I want model versions and configurations recorded, so that results are reproducible.	Should	Every reported result links to a model version and configuration.

3.3 Use Cases

	UC-01 Extract Menu from Image or PDF
Primary actor	Restaurant admin or menu manager
Preconditions	The user is authenticated and authorized; the extraction service is available.
Main flow	1. The user uploads a menu image or PDF file. 2. The system validates the file type, size, and readability. 3. The system extracts sections, item names, prices, and descriptions. 4. The system returns a structured draft with uncertainty flags.
Alternative or exception flows	2a. The file is unsupported or unreadable: the system shows a clear error and stops. 3a. Extraction fails or finds no menu content: the system reports this and offers manual entry.
Postconditions	A structured draft exists and awaits review; nothing is saved yet.

	UC-02 Review and Save Extracted Menu Data
Primary actor	Restaurant admin or menu manager
Preconditions	UC-01 completed and a draft exists.
Main flow	1. The system shows the draft next to the source file. 2. The user corrects, adds, or removes fields. 3. The user approves the draft. 4. The system saves the approved records to the EasyMenu menu.
Alternative or exception flows	3a. The user rejects the draft: nothing is saved. 4a. Saving fails: the system keeps the draft and shows an error.
Postconditions	Only approved records are stored; corrections are logged.

	UC-03 Ask an Analytics Question
Primary actor	Restaurant owner or manager
Preconditions	The user is authenticated; the database schema is configured; a read-only connection exists.
Main flow	1. The user enters a question in natural language. 2. The system supplies schema context and generates a candidate SQL query. 3. The validator checks that the query is a single read-only statement over allowed objects. 4. The system executes it with read-only credentials, a timeout, and a row limit. 5. The system explains the result in understandable language.
Alternative or exception flows	3a. Validation fails: the query is not run, the attempt is logged, and the user is told why (see UC-04). 4a. Timeout or error: the system reports the failure without partial output. 2a. The question is ambiguous or outside scope: the system asks for clarification or declines.
Postconditions	The user has an answer grounded in returned data; the query and outcome are logged.

	UC-04 Handle Unsafe or Unsupported Analytics Request
Primary actor	Restaurant owner or manager, system administrator
Preconditions	UC-03 is in progress.
Main flow	1. The validator detects a write operation, a disallowed object, multiple statements, or an out-of-scope request. 2. The system blocks execution. 3. The system shows a clear message and logs the event.
Alternative or exception flows	1a. The request is merely unsupported: the system suggests a supported question.
Postconditions	No unsafe statement reaches the database; an audit entry exists.

	UC-05 Generate Food Image
Primary actor	Menu manager
Preconditions	The user is authenticated; an item name and description are provided; the image service is available.
Main flow	1. The user submits the item name and description. 2. The system generates an image. 3. The system shows the image labelled as AI-generated.
Alternative or exception flows	2a. Generation fails or times out: the system shows a clear failure message and allows a retry.
Postconditions	A candidate image exists; it is not attached to the item until approved.

	UC-06 Review and Regenerate Food Image
Primary actor	Menu manager
Preconditions	UC-05 completed.
Main flow	1. The user inspects the image. 2. The user approves it, regenerates it, or rejects it. 3. On approval, the system attaches the image to the menu item.
Alternative or exception flows	2a. The user regenerates: the system returns to UC-05 step 2. 2b. The user rejects: no image is attached.
Postconditions	Only an approved image is attached; the model version and settings are recorded.

3.4 Functional Requirements

Components: A menu extraction, B analytics chatbot, C food image generation.

ID	Requirement	Comp.	Priority	Verification
FR-01	Accept menu images and PDF files; the supported formats are to be confirmed.	A	Must	Functional test
FR-02	Validate uploaded files and report unsupported, unreadable, or empty files clearly.	A	Must	Negative test
FR-03	Extract menu sections, item names, prices, and descriptions when present.	A	Must	Field-level evaluation
FR-04	Extract other supported fields, such as currency or size variants.	A	Could	Field-level evaluation
FR-05	Return structured output and flag uncertain or missing fields where the pipeline supports it.	A	Should	Inspection
FR-06	Let the user review, correct, and approve extracted data; save only approved data.	A	Must	UI and acceptance test
FR-07	Process multi-page PDF files and merge page results.	A	Should	Functional test
FR-08	Accept natural-language questions within the defined supported scope.	B	Must	Functional test
FR-09	Provide schema context to the model and generate a candidate SQL query.	B	Must	Test-set evaluation
FR-10	Validate each query (single read-only statement, allowed objects) before execution.	B	Must	Safety suite
FR-11	Execute queries only with read-only credentials, a timeout, and a row limit.	B	Must	Safety suite and configuration review
FR-12	Block unsafe, unauthorized, or unsupported requests with an understandable message.	B	Must	Safety suite
FR-13	Explain results in natural language grounded in the returned data, and state when a question cannot be answered reliably.	B	Must	Manual review
FR-14	Optionally display the generated SQL to the user for transparency.	B	Could	UI test
FR-15	Accept an item name and description and return a generated image or a clear failure message.	C	Must	Functional test
FR-16	Let the user review, regenerate, or reject images; attach only approved images.	C	Must	Acceptance test
FR-17	Label generated images as AI-generated.	C	Should	Inspection
FR-18	Offer optional style or negative-prompt controls.	C	Could	Functional test
FR-19	Log relevant events (errors, model version, blocked SQL) without secrets or unnecessary sensitive data.	All	Must	Log inspection

ID	Requirement	Comp.	Priority	Verification
FR-20	Handle model errors, timeouts, and unavailable dependencies with understandable messages.	All	Must	Fault-injection test
FR-21	Expose each AI capability through a documented interface for EasyMenu; the interface design is pending.	All	Must	Integration test
FR-22	Provide setup, configuration, usage, and testing instructions in the repository.	All	Must	Documentation review
FR-23	Record the model version and configuration with each output.	All	Should	Log inspection
FR-24	Restrict the features to authenticated and authorized EasyMenu users through the host system.	All	Must	Security test

3.5 Non-functional Requirements

Numeric targets are proposed and must be validated against measurements before they are treated as requirements.

ID	Category	Requirement and criterion	Verification
NFR-01	Security and access control	Secrets in environment variables or a secret manager; uploads validated; authorization enforced by the host system; database privileges limited to read-only for analytics.	Security checklist and review
NFR-02	Privacy and data protection	Use only permitted data; remove personal or confidential information from datasets; avoid logging sensitive menu or customer data unnecessarily.	Privacy review
NFR-03	Reliability and error handling	Invalid inputs, model errors, and timeouts produce understandable messages; target of at least 95% of valid test inputs completing (to be validated).	Test-run log
NFR-04	Performance and response time	Measure and report median and 95th-percentile latency per workflow on documented hardware; set targets after baseline measurement.	Timing measurements
NFR-05	Usability	Clear input guidance, understandable outputs, and a visible review step; a first-time user completes each main task without developer help.	Usability walkthrough
NFR-06	Maintainability	Separate inference, validation, and integration modules with consistent naming, comments, and error handling.	Code review
NFR-07	Scalability	Design so that each AI service can run and scale separately; no scale target is set for the prototype.	Architecture review
NFR-08	Compatibility	Interfaces compatible with the EasyMenu technology stack and supported browsers; both are to be confirmed.	Integration test
NFR-09	Auditability	Questions, generated SQL, validation outcomes, and approvals are logged for later review.	Log inspection
NFR-10	Resource efficiency	Document GPU memory and inference time; prefer parameter-efficient adaptation where feasible.	Measurement report
NFR-11	Reproducibility	Document model versions, datasets, preprocessing, configuration, evaluation methods, and random seeds where practical.	Experiment log audit
NFR-12	Licensing compliance	Use only models and data whose licences permit the intended use; keep a licence register (see Section 2).	Licence register review
NFR-13	Transparency of generated content	Generated images are identifiable as AI-generated; the system does not claim a generated image is a photograph.	Inspection

3.6 System Constraints, Assumptions, and Dependencies

Constraints

- The project must meet the official DEPI dates in Section 1.5, which leaves roughly six weeks for design, implementation, testing, and reporting.

- The team has five members, within the four to six allowed by the DEPI guidelines.
- Compute resources, dataset sizes, and licences limit what can be trained or adapted.
- A production deployment and production-scale performance are outside the project scope.

Assumptions and dependencies

- The EasyMenu codebase and database schema will be available to authorized team members for integration and testing.
- Final model selection depends on hardware, licences, representative data, and initial experiments.
- The system must not execute unvalidated model-generated SQL or publish extracted menu data without review.
- Dataset sizes, exact model versions, deployment target, quantitative targets, and final assignments remain to be confirmed by the team and mentor.

3.7 Requirements Prioritization

The table summarises the functional requirements by component and MoSCoW priority.

Component	Must	Should	Could	Total
A. Menu extraction	4	2	1	7
B. Analytics chatbot	6	0	1	7
C. Food image generation	2	1	1	4
Cross-cutting	5	1	0	6
Total	17	4	3	24

Must requirements define the minimum viable version of each component. Should and Could requirements are scheduled only after the Must requirements of all three components are met.

3.8 Requirements Traceability Matrix

Objective	Requirements	Use cases	Comp.	Verification	KPI
O1 Menu extraction	FR-01 to FR-07; NFR-03, NFR-05	UC-01, UC-02	A	Field-level evaluation; acceptance test	Extraction F1; price match; correction effort
O2 Analytics chatbot	FR-08 to FR-14; NFR-01, NFR-09	UC-03, UC-04	B	Test set; safety suite; manual review	Execution accuracy; SQL safety; faithfulness
O3 Image generation	FR-15 to FR-18; NFR-10, NFR-13	UC-05, UC-06	C	Human rubric; acceptance test	Prompt adherence; acceptability
O4 Integration	FR-19 to FR-24; NFR-04, NFR-06, NFR-08	All	All	Integration test; code review	Integration completeness; response time
O5 Safety and quality	FR-10 to FR-12, FR-20; NFR-01 to NFR-03	UC-04	All	Safety suite; fault-injection test	SQL safety; reliability
O6 Deliverables	FR-22; NFR-11, NFR-12	—	All	Documentation review; log audit	Reproducibility; on-time delivery

Appendix A. Items Requiring Team Confirmation

The following facts are not available in the source files and are therefore not assumed in this document.

#	Item	Why it matters
1	Which submission date applies: 19 October 2026 (team deadline) or 16 October 2026 (DEPI guidelines).	Compliance with official dates.
2	Final role assignments and the mentor's name.	Section 1.8 is a proposal; Section 2.7 awaits feedback.
3	Which third-party AI/API services EasyMenu currently uses.	Defines the baseline the project aims to replace or reduce.
4	EasyMenu technology stack, database engine, schema, and how restaurants' data are separated.	Determines integration, SQL dialect, and access control.
5	Menu languages to support (Arabic and English are candidates) and whether PDFs are born-digital or scanned.	Model choice and evaluation data for Component A.
6	Sources and permissions for sample menus, question–SQL data, and food images; dataset sizes.	Data availability, licensing, and privacy.
7	Available GPU compute and the demonstration environment.	Feasibility of adaptation and deployment.
8	Final model choices after baselines, including a current licence check (for example LayoutLMv3 and SDXL).	Legal use and reproducibility.
9	Quantitative targets for the KPIs, agreed with the mentor.	Section 1.10 values are proposals.
10	Official grading rubric, if one is supplied.	Section 2.8 weights are a working assumption.

References

Each source below was looked up in public sources on 19 October 2026. "Verified" means the title, authors, venue, and identifier match the retrieved record; any open point is stated in the entry.

[1] PaddlePaddle, "PaddleOCR," GitHub repository. https://github.com/PaddlePaddle/PaddleOCR (accessed 19 Oct 2026). Verified: repository and release notes consulted.

[2] PaddleOCR 3.0 Technical Report, arXiv:2507.05595, July 2025. https://arxiv.org/abs/2507.05595. Verified; author list not transcribed here, see the arXiv entry.

[3] Y. Huang, T. Lv, L. Cui, Y. Lu, and F. Wei, "LayoutLMv3: Pre-training for Document AI with Unified Text and Image Masking," in Proc. 30th ACM Int. Conf. Multimedia (ACM MM), 2022. arXiv:2204.08387. https://arxiv.org/abs/2204.08387. Verified.

[4] Microsoft, "layoutlmv3-base," Hugging Face model card. https://huggingface.co/microsoft/layoutlmv3-base (accessed 19 Oct 2026). Verified; the listed licence (CC BY-NC-SA 4.0) has changed in the card history and must be re-checked before use.

[5] G. Kim et al., "OCR-free Document Understanding Transformer," in Computer Vision – ECCV 2022. arXiv:2111.15664. DOI: 10.1007/978-3-031-19815-1_29. https://arxiv.org/abs/2111.15664. Verified.

[6] T. Yu et al., "Spider: A Large-Scale Human-Labeled Dataset for Complex and Cross-Domain Semantic Parsing and Text-to-SQL Task," in Proc. EMNLP 2018, pp. 3911–3921. arXiv:1809.08887. https://aclanthology.org/D18-1425/. Verified.

[7] B. Wang, R. Shin, X. Liu, O. Polozov, and M. Richardson, "RAT-SQL: Relation-Aware Schema Encoding and Linking for Text-to-SQL Parsers," in Proc. ACL 2020. DOI: 10.18653/v1/2020.acl-main.677. arXiv:1911.04942. https://arxiv.org/abs/1911.04942. Verified.

[8] B. Rozière et al., "Code Llama: Open Foundation Models for Code," arXiv:2308.12950, 2023 (v3, Jan 2024). https://arxiv.org/abs/2308.12950. Verified.

[9] J. Ho, A. Jain, and P. Abbeel, "Denoising Diffusion Probabilistic Models," in Advances in Neural Information Processing Systems 33 (NeurIPS 2020). arXiv:2006.11239. https://arxiv.org/abs/2006.11239. Verified.

[10] R. Rombach, A. Blattmann, D. Lorenz, P. Esser, and B. Ommer, "High-Resolution Image Synthesis with Latent Diffusion Models," in Proc. IEEE/CVF CVPR 2022, pp. 10684–10695. arXiv:2112.10752. https://arxiv.org/abs/2112.10752. Verified.

[11] D. Podell et al., "SDXL: Improving Latent Diffusion Models for High-Resolution Image Synthesis," in Proc. ICLR 2024. arXiv:2307.01952. https://arxiv.org/abs/2307.01952. Verified.

[12] Stability AI, "stable-diffusion-xl-base-1.0," Hugging Face model card. https://huggingface.co/stabilityai/stable-diffusion-xl-base-1.0 (accessed 19 Oct 2026). Verified: CreativeML Open RAIL++-M licence and stated limitations.

[13] E. J. Hu et al., "LoRA: Low-Rank Adaptation of Large Language Models," in Proc. ICLR 2022. arXiv:2106.09685. https://arxiv.org/abs/2106.09685. Verified.

Project resources: EasyMenu AI repository, https://github.com/mohanadakl/EasyMenu-AI (accessed 19 Oct 2026); DEPI Project Documentation Guidelines (2026), supplied by the team.